

# Pandas and NumPy Basics for CIVL6410

Jiwon Kim, The University of Queensland

2026

## Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>Pandas and NumPy Basics for CIVL6410</b>                 | <b>2</b> |
| 1. NumPy Essentials                                         | 2        |
| Infinity                                                    | 2        |
| Zeros array                                                 | 2        |
| Basic maths                                                 | 2        |
| Key points                                                  | 2        |
| 2. Creating a Pandas DataFrame                              | 2        |
| 2.1. From a dictionary                                      | 3        |
| Key points                                                  | 3        |
| 2.2. From a CSV file                                        | 4        |
| Key points                                                  | 4        |
| Try it yourself                                             | 5        |
| 3. Inspecting a Pandas DataFrame                            | 5        |
| Key points                                                  | 6        |
| Try it yourself                                             | 6        |
| 4. Accessing DataFrame Cells with <code>.at[]</code>        | 6        |
| Key points                                                  | 6        |
| 5. Setting an Index in DataFrames                           | 6        |
| Single-column index                                         | 6        |
| Two-column (tuple) index                                    | 7        |
| Key points                                                  | 8        |
| Try it yourself                                             | 8        |
| 6. Adding and Updating DataFrame Columns                    | 8        |
| Key points                                                  | 8        |
| Try it yourself                                             | 9        |
| 7. Iterating DataFrame Rows with <code>.itertuples()</code> | 9        |
| Key points                                                  | 10       |
| Try it yourself                                             | 10       |
| Quick Reference                                             | 10       |

# Pandas and NumPy Basics for CIVL6410

## 1. NumPy Essentials

NumPy (numpy) provides numerical tools. In this course the two things you need most are `np.inf` and basic array operations.

### Infinity

```
import numpy as np

# np.inf represents positive infinity
x = np.inf
print(x)          # inf
print(x + 100)    # inf    (infinity + anything = infinity)
print(x > 9999)   # True

# Comparison - used to check if a variable has been processed yet
label = np.inf    # initialise a variable's value to infinity (arbitrarily large value)
if label == np.inf:
    print("not yet processed")
```

### Zeros array

```
# Create an array of zeros (useful for initialising values)
arr = np.zeros(5)
print(arr)          # [0. 0. 0. 0. 0.]
print(arr[0])       # 0.0
arr[2] = 7.5
print(arr)          # [0. 0. 7.5 0. 0.]
```

### Basic maths

```
print(np.sqrt(16))    # 4.0
print(np.round(3.14159, 2)) # 3.14
print(np.abs(-5.5))   # 5.5
```

### Key points

- Use `np.inf` when you need “infinity” (e.g., initial label values in shortest-path algorithms).
- `np.zeros(n)` creates an array of `n` zeros.

## 2. Creating a Pandas DataFrame

**pandas** (`pd`) is Python’s library for working with tabular data. It is built on top of NumPy and provides two core data structures:

- A **Series** is a single column of data — like a labelled list.
- A **DataFrame** is a two-dimensional table with labelled rows and columns — the pandas equivalent of a spreadsheet or a database table.

In this course you will mostly work with DataFrames. A DataFrame stores road network data (links, nodes, OD pairs) in a structured way that makes it easy to filter, sort, compute new columns, and export results.

Every column in a DataFrame can hold a different data type (integers, floats, strings, etc.), and every row has an **index** — a label used to identify it, which by default is 0, 1, 2, ....

There are two common ways to create a DataFrame: from a Python dictionary written in code, or by reading a CSV file from disk. Both are shown below.

## 2.1. From a dictionary

A simple way to create a DataFrame in code is from a **dictionary**, where each key is a column name and each value is a list of column values.

```
import pandas as pd

# Create a DataFrame from a dictionary
data = {
    'name':    ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'subject': ['Maths', 'Maths', 'Physics', 'Physics', 'Maths'],
    'score':   [88, 75, 92, 61, 95],
}

df = pd.DataFrame(data)
print(df)
#      name  subject  score
# 0  Alice    Maths    88
# 1   Bob    Maths    75
# 2  Carol  Physics    92
# 3   Dave  Physics    61
# 4   Eve    Maths    95
```

You can also create a DataFrame from a list of dictionaries (one dict per row):

```
rows = [
    {'product': 'apple', 'price': 1.20, 'stock': 50},
    {'product': 'banana', 'price': 0.50, 'stock': 120},
    {'product': 'cherry', 'price': 3.80, 'stock': 30},
]

df_products = pd.DataFrame(rows)
print(df_products)
#   product  price  stock
# 0  apple    1.20     50
# 1 banana    0.50    120
# 2  cherry    3.80     30
```

## Key points

- A **DataFrame** is a table with labelled rows and columns — like a spreadsheet in Python.
- Each column can hold a different data type (int, float, string, etc.).
- The prefix `df_` is a common naming convention for DataFrames.

## 2.2. From a CSV file

### What is a CSV file?

A **CSV** (Comma-Separated Values) file is a plain text file that stores a table. Each line is a row, and the values in each row are separated by commas. The first line is usually the header (column names).

Here is the student data from above written as a CSV file. You can create this file yourself in any text editor (including VS Code) and save it as `scores.csv`:

```
name,subject,score
Alice,Maths,88
Bob,Maths,75
Carol,Physics,92
Dave,Physics,61
Eve,Maths,95
```

### Saving a DataFrame to a CSV file

If you already have a DataFrame in Python, you can write it to a CSV file with `.to_csv()`:

```
# Save df to a file called scores.csv
# index=False prevents pandas from writing the row numbers as an extra column
df.to_csv('scores.csv', index=False)
```

Open `scores.csv` in a text editor or Excel to verify it looks exactly like the CSV shown above.

### Loading a CSV file into a DataFrame

Use `pd.read_csv()` to load a CSV file into a DataFrame. The first argument is the file path. The `sep` parameter tells pandas what character separates the values in each row — for a standard CSV file this is a comma `,`, but some files use a semicolon `;` or a tab `\t` instead. If you omit `sep`, pandas assumes `,` by default.

The file path is **relative to the working directory** — the folder Spyder is currently set to (shown in the toolbar dropdown). If `scores.csv` is in the same folder as your script, just write the filename. If it is in a subfolder called `data/`, write `'data/scores.csv'`. Avoid absolute paths like `/Users/name/project/data/scores.csv` — they break when the project is moved to another machine.

```
# Load scores.csv back into a new DataFrame
df2 = pd.read_csv('scores.csv', sep=',')
print(df2)
#      name  subject  score
# 0  Alice    Maths    88
# 1   Bob    Maths    75
# 2  Carol  Physics    92
# 3   Dave  Physics    61
# 4   Eve    Maths    95
```

The result is identical to the DataFrame we created from the dictionary. From this point on, it doesn't matter how the data was created — you work with the DataFrame the same way.

## Key points

- `df.to_csv('filename.csv', index=False)` saves a DataFrame. Pass `index=False` if you do not want an extra row number column in the file.
- `pd.read_csv('filename.csv', sep=',')` loads a CSV file into a DataFrame.

- The file path is **relative to the folder where your Python script is**. If your script is in the same folder as the CSV file, just use the filename. If the CSV is in a different folder, you need to include the relative path (e.g. `data/scores.csv`).

## Try it yourself

1. Create a DataFrame with columns 'city', 'country', and 'population' for at least three cities of your choice. Print it.
2. Save it to `cities.csv` with `index=False`. Open the file in a text editor and verify the content.
3. Load `cities.csv` back into a new DataFrame and print it.

## 3. Inspecting a Pandas DataFrame

```
data = {
    'name':    ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'subject': ['Maths', 'Maths', 'Physics', 'Physics', 'Maths'],
    'score':   [88, 75, 92, 61, 95],
}
df = pd.DataFrame(data)

# Column names
print(df.columns)
# Index(['name', 'subject', 'score'], dtype='object')

# Data types of each column
print(df.dtypes)
# name      object
# subject   object
# score      int64
# dtype: object

# Row index (default: 0, 1, 2, ...)
print(df.index)
# RangeIndex(start=0, stop=5, step=1)

# Number of rows and columns
print(df.shape)    # (5, 3)

# First few rows
print(df.head(3))
#      name  subject  score
# 0  Alice    Maths    88
# 1   Bob    Maths    75
# 2  Carol  Physics    92

# Quick summary (prints to console)
df.info()
```

## Key points

- `.dtypes` tells you whether a column holds integers, floats, or strings — important when doing arithmetic.
- `.shape` returns (`rows`, `columns`) as a tuple.
- `object` dtype means the column holds strings.

## Try it yourself

1. Print the number of rows and columns in `df_products`.
2. Print the data types of `df_products`.

## 4. Accessing DataFrame Cells with `.at[]`

`.at[row_label, column_name]` reads or writes a **single cell** by its row label and column name.

With the default index (0, 1, 2, ...), the row label is just the row number:

```
data = {
    'name':    ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'subject': ['Maths', 'Maths', 'Physics', 'Physics', 'Maths'],
    'score':   [88, 75, 92, 61, 95],
}
df = pd.DataFrame(data)

# Read a single value
print(df.at[0, 'name'])    # Alice
print(df.at[2, 'score'])   # 92
print(df.at[4, 'subject']) # Maths

# Write (update) a single value
df.at[1, 'score'] = 80
print(df.at[1, 'score'])   # 80
```

## Key points

- `.at[]` is the fastest way to read/write a single value.
- The row label must match the current index. After calling `.set_index()` (next section), the row labels change.

## 5. Setting an Index in DataFrames

By default rows are numbered 0, 1, 2, ... It is often more useful to set a meaningful column as the index so you can look up rows by a real identifier — e.g. student name, or product ID.

### Single-column index

```
data = {
    'name':    ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'subject': ['Maths', 'Maths', 'Physics', 'Physics', 'Maths'],
    'score':   [88, 75, 92, 61, 95],
}
```

```

}
df = pd.DataFrame(data)

df.set_index('name', inplace=True, drop=False)
# inplace=True - modify df directly (no need to reassign)
# drop=False - keep 'name' as a regular column as well

print(df)
#           name  subject  score
# name
# Alice  Alice    Maths    88
# Bob    Bob     Maths    75
# Carol  Carol   Physics    92
# Dave   Dave   Physics    61
# Eve    Eve     Maths    95

# Now look up by name directly:
print(df.at['Carol', 'score'])    # 92
print(df.at['Dave', 'subject'])   # Physics

# Check if a row exists
print('Alice' in df.index)        # True
print('Zara' in df.index)         # False

```

## Two-column (tuple) index

Sometimes a row is identified by a combination of two columns. Set both as the index — then look up using a tuple.

```

results = {
    'student': ['Alice', 'Alice', 'Bob', 'Bob', 'Carol'],
    'subject': ['Maths', 'Physics', 'Maths', 'Physics', 'Maths'],
    'score':   [88, 74, 75, 83, 92],
}
df_results = pd.DataFrame(results)

df_results.set_index(['student', 'subject'], inplace=True, drop=False)

print(df_results)
#           student  subject  score
# student subject
# Alice  Maths    Alice    Maths    88
#        Physics  Alice    Physics    74
# Bob    Maths    Bob     Maths    75
#        Physics  Bob     Physics    83
# Carol  Maths    Carol    Maths    92

# Look up Alice's Physics score using a tuple:
print(df_results.at[('Alice', 'Physics'), 'score'])    # 74
print(df_results.at[('Bob', 'Maths'), 'score'])        # 75

# Check existence
print(('Alice', 'Maths') in df_results.index)          # True
print(('Alice', 'Biology') in df_results.index)        # False

```

## Key points

- `inplace=True` modifies the DataFrame in place — without it you would need `df = df.set_index(...)`.
- `drop=False` keeps the indexed columns accessible by name with `.at[]`.
- With a tuple index, always pass a tuple for the row label: `df.at[('Alice', 'Maths'), 'score']`.

## Try it yourself

1. Using `df` (student/score table), set 'name' as the index and look up Eve's score.
2. Using `df_results`, check whether ('Carol', 'Physics') exists in the index.

## 6. Adding and Updating DataFrame Columns

```
data = {
    'name':    ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'subject': ['Maths', 'Maths', 'Physics', 'Physics', 'Maths'],
    'score':   [88, 75, 92, 61, 95],
}
df = pd.DataFrame(data)
df.set_index('name', inplace=True, drop=False)

# Add a new column and fill every row with the same value
df['passed'] = False
print(df)
#      name  subject  score  passed
# name
# Alice  Alice   Maths    88   False
# ...

# Update a single cell
df.at['Alice', 'passed'] = True
print(df.at['Alice', 'passed'])    # True

# Add a column computed from an existing column
df['score_out_of_10'] = df['score'] / 10
print(df[['score', 'score_out_of_10']])
#      score  score_out_of_10
# name
# Alice    88             8.8
# Bob      75             7.5
# ...

# Add a column with a constant string value
df['semester'] = 'S1 2026'
print(df.columns)
# Index(['name', 'subject', 'score', 'passed', 'score_out_of_10', 'semester'], dtype='object')
```

## Key points

- `df['new_col'] = value` adds a column and fills every row with `value`.



- `df['new_col'] = df['existing_col'] * factor` creates a column by applying arithmetic row-by-row.
- Use `.at[row_label, 'col']` to update a single cell after the column exists.

## Try it yourself

1. Add a column 'grade' to `df` initialised to '' (empty string).
2. Set `df.at['Carol', 'grade'] = 'HD'` and print the result.

## 7. Iterating DataFrame Rows with `.itertuples()`

`.itertuples()` lets you loop over every row of a DataFrame. Each row is returned as a named tuple — you access columns by name using dot notation. **pandas** also has `.iterrows()` for iterating rows, but it returns each row as a Series (not a tuple) and is much slower than `.itertuples()`, so we will use `.itertuples()` in this course.

```
data = {
    'name': ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'score': [88, 75, 92, 61, 95],
}
df = pd.DataFrame(data)
df.set_index('name', inplace=True, drop=False)

# index=False - each row is a tuple of column values only
for row in df.itertuples(index=False):
    print(row)
# Pandas(name='Alice', score=88)
# Pandas(name='Bob', score=75)
# ...

# Access individual fields by column name
for row in df.itertuples(index=False):
    print(f'{row.name} scored {row.score}')
# Alice scored 88
# Bob scored 75
# ...

# index=True - row.Index is the row label (here: the student name)
for row in df.itertuples(index=True):
    print(f'index={row.Index}, name={row.name}, score={row.score}')
# index=Alice, name=Alice, score=88
# index=Bob, name=Bob, score=75
# ...

# Typical write-back pattern: compute a new value and store it in the DataFrame
df['grade'] = ''

for row in df.itertuples(index=True):
    if row.score >= 85:
        df.at[row.Index, 'grade'] = 'HD'
    elif row.score >= 65:
        df.at[row.Index, 'grade'] = 'D'
    elif row.score >= 50:
```

```

        df.at[row.Index, 'grade'] = 'C'
    else:
        df.at[row.Index, 'grade'] = 'F'

print(df)
#           name  score grade
# name
# Alice  Alice     88    HD
# Bob    Bob      75     D
# Carol  Carol     92    HD
# Dave   Dave     61     C
# Eve    Eve      95    HD

```

## Key points

- Always prefer `itertuples()` over `iterrows()` — it is significantly faster and gives typed access to columns.
- `row.Index` holds the current row label. Use it with `.at[row.Index, 'col']` to write back a value.
- `index=False` is useful when you only need to read values and don't need to write back.
- With a tuple index, `row.Index` is a tuple (e.g. ('Alice', 'Maths')). It works directly in `.at[row.Index, 'col']`.

## Try it yourself

1. Use `itertuples(index=False)` to print each student as "Alice: 88/100".
2. Use `itertuples(index=True)` to add a column 'bonus' where students scoring  $\geq 90$  get 5, everyone else gets 0.

## Quick Reference

```

import pandas as pd
import numpy as np

np.set_printoptions(legacy='1.25')

# --- Create DataFrame ---
df = pd.DataFrame({'col_a': [1, 2, 3], 'col_b': ['x', 'y', 'z']})

# --- Load from CSV ---
df = pd.read_csv('file.csv', sep=',')

# --- Inspect ---
print(df)
print(df.dtypes)
print(df.shape)      # (rows, cols)
print(df.columns)
print(df.head(3))

# --- Set index ---
df.set_index('id_col', inplace=True, drop=False)           # single column

```

```

df.set_index(['col_a', 'col_b'], inplace=True, drop=False)      # tuple index

# --- Access / update single cell ---
val = df.at[row_label, 'col']
df.at[row_label, 'col'] = new_value

# --- Check if a row label exists ---
print(label in df.index)

# --- Add a column ---
df['new_col'] = 0.0      # constant
df['new_col'] = df['other_col'] * 2      # computed

# --- Iterate rows (read only) ---
for row in df.itertuples(index=False):
    print(row.col_name)

# --- Iterate rows (read + write back) ---
for row in df.itertuples(index=True):
    df.at[row.Index, 'new_col'] = row.col_name * 2

# --- NumPy ---
np.inf      # positive infinity
np.zeros(n) # array of n zeros
np.sqrt(x)  # square root
np.round(x, 2) # round to 2 decimal places

```